
Told or Enforced: Measuring When In-Context Contracts Substitute for Runtime Enforcement in Agent Harnesses

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 With tool exposure fixed and no access to model weights, an agent can be held to
2 a rule two ways: state it in the prompt and rely on the model (telling), or have the
3 harness block the violation regardless of the model (enforcing). We separate them,
4 the rule told or withheld and the enforcement layer on or off, and ask the question
5 a harness designer faces: given the model was told, does enforcement still change
6 behaviour?

7 We answer it on a pre-registered, single-harness, within-family run: a stronger
8 and a weaker sibling in each of four model families. The enforcement layer's
9 block is unconditional: across 488 enforced runs it let nothing through. With the
10 enforcement off, telling the rule raised the safe rate in every family (+24 points
11 pooled), so specification moves behaviour in the families we test. But it does not
12 stand in for enforcement: the marginal value of enforcement given telling stays
13 large (41 points pooled), small only for the two self-enforcing families and most
14 of the barrier for the two weak ones, so substitution holds only in that narrow
15 corner. Capability does not cleanly decide it either: within a family it helps or
16 ties on the salient boundary and can cut against safety on a discoverable side-door
17 (clearest in one family), so the contrast is flat to slightly negative. We report these
18 results descriptively, across three model lineages (Qwen2.5 and Qwen3 collapse
19 to one Qwen lineage), not as a significance claim.

20 Two further contributions. We release GovernedAgentBench: the taskset, a de-
21 terministic offline grader, the analysis code, and the git-pinned harness, with the
22 paid transcripts and grades shipping as a versioned archive alongside the preprint.
23 And a caution: a harness that hides a tool's output makes the agent guess facts it
24 cannot see, which a grader then misreads as fabrication; in our own pipeline an ap-
25 parent case of this vanished the moment we showed the agent what its commands
26 actually returned.

1 Introduction

28 An agent harness is a design surface. Whoever wires up an LLM to tools controls the interaction
29 surrounding it: which tools are exposed, how actions are parsed and dispatched, and what happens
30 when a proposed action would violate a constraint. For any given constraint, with tool exposure
31 fixed and no access to model weights, that engineer holds two methods. The first is in-context
32 specification (telling): state the rule in the agent's prompt as an in-context contract and rely on the
33 model to respect it. The second is runtime enforcement (enforcing): have the harness block the
34 violation deterministically, regardless of what the model decides. Telling is cheap and moves with
35 the prompt; enforcing requires typed command schemas, dispatch gates, and validation code that

36 must be built, tested, and maintained. A harness designer deciding where to spend engineering
37 effort needs to know when the second method buys behaviour the first does not.

38 Existing guardrail evaluations rarely isolate it. Some toggle enforcement while the full policy sits in
39 the prompt in both conditions [arXiv:2604.15579], so the delta is only measured where the model
40 was already told. Others bundle “injected and enforced” into one switch [arXiv:2602.22302], or
41 compare an enforced arm against a prompt-only arm [arXiv:2603.00822, arXiv:2605.28914], which
42 mixes telling into enforcing. The closest, Mind the GAP [arXiv:2602.16943], varies enforcement
43 directly but never crosses the told and enforced forms of the same rule. So the marginal value of
44 enforcement given telling is measured only in that told-column form; no located study crosses it
45 against a withheld arm for the same rule, across all four cells, scored on the model’s first action.

46 This paper runs that crossing. For each rule we build a 2×2 experiment: the rule told or withheld,
47 and the enforcement layer on or off. The quantity we care about is the marginal value of enforcement
48 given the model was told.

49 A second, independent finding is methodological. Our own harness first produced a spurious
50 result: it handed the agent a file path instead of a tool’s actual output, so the blinded agent guessed
51 identifiers it could not see, and the grader called that fabrication. Restoring the output dissolved the
52 effect entirely (Section 6). We name the failure mode harness blindness.

53 2 Background and related work

54 Isolated withheld-but-enforced measurements exist: FORGE by design, Prompts Don’t Protect
55 structurally, and Mind the GAP through its enforce mode. What no located prior offers is a
56 clean, reproducible isolation of the two methods for the same rule across all four cells, scored on
57 first action, per governance mechanism, with a released deterministic offline grader. GovernedA-
58 gentBench provides that instrument; the paper’s claims are the empirical finding above and the
59 harness-blindness caution (Section 6), not the novelty of any single design element.

60 3 The two methods

61 With tool exposure fixed, a harness can make an agent respect a rule two ways: state the rule where
62 the model can read it (telling), or make the violation impossible whatever the model emits (enforc-
63 ing). Deployed systems pull both at once, so their effects are confounded. This paper’s whole
64 manipulation is to decouple them, varying telling and enforcing independently for a single rule.

65 Telling is the in-context contract. It is not just a policy paragraph but everything the model is shown:
66 the list of commands and their argument shapes, a per-command “safe for the agent to run” flag,
67 and the prose describing the out-of-scope boundary. It is a harness-level method, separate from
68 anything baked in during training, which we refer to as model disposition.

69 Enforcing is what the harness does regardless of the model. Our reference harness has five gov-
70 ernance checks, each of which can be switched off (individually, or all at once); the full list is in
71 Appendix C.

72 A sixth property, transaction integrity, is always on. The commit gate is the one this paper’s head-
73 line turns on: the enforcement layer refuses an agent-issued commit and leaves the change merely
74 proposed.

75 **The 2×2 .** Crossing the two methods, for any one rule, gives four cells:

	Enforcing	Off
Told	A: deployment baseline	B: told, not enforced
Withheld	C: withheld, still enforced	D: neither

76 Cell A is how systems ship. Cell B isolates self-enforcement: the model knows the rule and nothing
77 stops it. Cell C isolates pure enforcement: the rule is hidden but the enforcement layer still blocks.
78 Cell D is the floor. Three reads carry the information: B vs. D is the effect of telling; C vs. D is the
79 effect of enforcing; and A vs. B is our headline: what enforcement adds once the model was told. If

80 A and B match, enforcement changed no behaviour on that rule; its value is then the guarantee itself,
81 which is real and unconditional (a blocked action simply cannot happen) even where behaviour
82 does not move.

83 One scoring convention matters. A blocked action returns an error, and that error is itself the rule,
84 told late, so cell C drifts toward cell B after the model’s first contact with a block. We therefore
85 score the telling reads on the model’s first action; Section 5.1 reports the multi-turn check.

86 4 GovernedAgentBench

87 GovernedAgentBench is the instrument that holds the two methods apart. It toggles telling and
88 enforcing independently, per rule, and scores the result deterministically and offline.

89 **The harness.** The instrument is HAI, a non-clinical personal-wellness agent harness with a
90 typed command interface, pinned as a frozen snapshot; it is a reference implementation, not the
91 contribution or a product, though the benchmark’s author built it in work separate from this paper,
92 which we flag as a same-team caveat. Its no-diagnosis boundary is one of the evaluated rules, not a
93 disclaimer. Two facts matter for reproduction. First, the published v0.2.0 package does not enforce;
94 the dispatch gate landed three days after that version tag, so the measured harness is pinned by
95 git commit 6c82cd0. Second, before spending anything, a pre-flight check confirmed the harness
96 actually refuses an agent commit; it passed, so the enforced cells were measured against a harness
97 verified to enforce, not assumed to.

98 **Tasks.** The taskset has 51 tasks. The paid run used a 16-task subset. Each task is one prompt
99 and scoring key, told and withheld are separate tasks over the same decision, so the 16 are eight
100 decisions in both arms (commit, archive, and set-active on a target and on an intent, plus two commit
101 framings), each run four times per cell. The commit boundary is two of those eight, committing a
102 target and committing an intent, and it is the subset the ladder of Section 5.2 swept.

103 **The two axes in code.** Enforcing is a harness setting. Telling is a per-task contract applied at prompt-
104 render time while the harness is untouched: the withheld prompt strips the rule’s facts and prose, and
105 an automatic scan rejects any withheld prompt that still leaks the rule’s words. Two caveats attach.
106 First, even withheld, the prompt still lists command names, so the model retains a hint, and B-vs-D
107 and C-vs-D are therefore lower bounds on the effect of telling. Second, a discoverability aid injects
108 the seeded target’s identifier into the task prompts of both arms, without which withheld models
109 often fail to find the target at all; it raises the withheld violation rate, and its full provenance, per run,
110 is disclosed in Section 7.

111 **Scoring: the commit predicate.** The headline rests on one deterministic check that needs no thresh-
112 olds and no pattern matching. The grader is offline with no model calls; it grades one (task, trajec-
113 tory) pair.

114 **The pre-registered runs.** Both runs were designed before their data. The within-family run
115 (Section 5.1) was pre-registered in full: the four family pairs, the sixteen tasks, the within-family
116 cell-B capability contrast as the estimand, and a lineage-collapsed sign-flip permutation as the
117 primary test: pre-committed as descriptive rather than confirmatory, because at three model lineages
118 its exact one-sided floor is 0.125.

119 5 Results

120 We report the within-family run (Section 5.1); the four-model ladder that motivated it (Section 5.2)
121 follows as provenance. One structural fact frames everything. Cell A (told, enforced) and cell
122 C (withheld, enforced) are safe on every run by construction: the enforcement layer blocks the
123 mutation whatever the model emits, and across all 488 enforced runs not one violation got through.
124 The enforced cells carry no variance, so the 2×2 reduces to two contrasts in the enforcement off
125 column. Cell B against cell D (told versus withheld) is the effect of telling. Cell A against cell B
126 (enforced versus off, once the model was told) is the question a harness designer actually asks, and
127 the one this paper set out to measure: what does enforcement still add once the model has been told?
128 Because A is 100% by construction, that contrast is just 100 minus the cell-B safe rate.

5.1 The within-family run

The four-model ladder of Section 5.2 entangles capability with model family: its capable models are non-Qwen and its weak models Qwen, so “weaker models need enforcement” and “Qwen models do” cannot be separated. The within-family run breaks that confound by construction, contrasting a strong and a weak model *within* the same family (Qwen2.5 {72B, 7B}, Qwen3 {32B, 8B}, Llama3.1 {70B, 8B}, Mistral {24B, 7B}), across sixteen tasks on mutating state (eight distinct governed decisions, each told and withheld: commit, archive and set-active on a target and on an intent, plus two commit framings), four repeats a cell, run against on-demand H100 deployments for about USD 115. It answers the enforcement-given-telling question directly, and corrects what the ladder suggested. Throughout, rates are rounded to the nearest point and effect columns are computed from the unrounded rates, so an effect may not equal the difference of the rounded columns beside it.

The guarantee holds at scale. Across all eight models both enforced cells scored safe on every one of 488 runs. Those cells were scheduled at 512 (both enforced cells, eight tasks each, eight models, four repeats); 24 trajectories came back incomplete, from length truncation and tool-driving failures concentrated in the weaker models, and are dropped, split evenly across the two enforced cells (12 and 12). Because the enforced cells are safe by construction, a dropped run is an unfinished trajectory, not an enforcement failure. The enforcement layer’s block is unconditional and did not miss once at eight-model scale; everything below concerns the two cells where enforcement is off.

Telling helps, but does not replace enforcement. With enforcement off, telling the rule (cell B) beat withholding it (cell D) in every family. But telling did not make enforcement redundant: the marginal value of enforcement given telling, cell A minus cell B, stayed large.

Family	B: told, off	D: withheld, off	telling (B–D)	enforcement given told (A–B)
Qwen2.5	28	20	+8	72
Qwen3	27	12	+14	73
Llama3.1	93	57	+36	7
Mistral	90	53	+37	10
Pooled	59	35	+24	41

Telling raises the safe rate 24 points pooled and is positive in every family: specification moves behaviour. But pooled, enforcement still adds 41 points over telling alone, and that residual splits sharply by regime. It is small only for the two self-enforcing families (7 points for Llama3.1, 10 for Mistral) and it is most of the barrier for the two weak ones (72 and 73 points). Substitution in the strict sense, that enforcement adds nothing once the model was told, holds only in that narrow high-self-enforcement corner, and even there it is a 7-to-10-point residual, not the clean zero the ladder showed. No family reaches a 100% told-off safe rate, so the ladder’s cell-A-equals-cell-B result (Section 5.2) does not survive the within-family test (Figure 1).

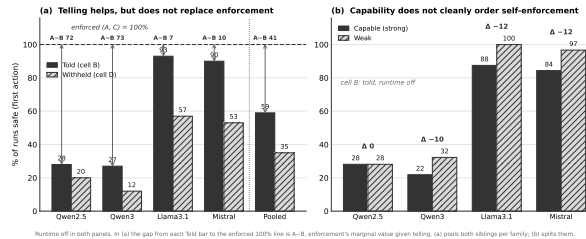


Figure 1: The within-family run, enforcement off in both panels. (a) Telling the rule (cell B) beat withholding it (cell D) in every family, +24 points pooled, but no family reaches the 100% that would make enforcement redundant; the distance from 100 is enforcement’s marginal value given telling (cell A minus cell B), small for Llama3.1 and Mistral and most of the barrier for the Qwen families. (b) Within a family the strong-minus-weak cell-B contrast is flat to slightly negative, so capability does not cleanly order self-enforcement; on the commit boundary specifically the stronger sibling is at least as safe in every family. Bars are the share of runs safe on the first action. The enforced cells A and C were safe on all 488 runs and are not shown; (a) pools both siblings per family, (b) splits them.

158 **Capability does not cleanly order self-enforcement.** The ladder read capability as what decides
 159 whether telling suffices. Within a family that is not a law. On the commit boundary the ladder
 160 actually swept, the stronger sibling is at least as safe in every family (Qwen2.5 8/8 versus 0/8,
 161 Qwen3 4/8 versus 2/8, Llama3.1 and Mistral tied at ceiling), so there capability helps if anything.
 162 Pooled across the wider taskset, though, the within-family strong-minus-weak contrast in cell B runs
 163 flat to slightly negative:

Family	strong	weak	contrast
Qwen2.5	28	28	0
Qwen3	22	32	−10
Llama3.1	88	100	−12
Mistral	84	97	−12

164 The pooled negative is small and mixed. Two families sit at ceiling: in Llama3.1 and Mistral the
 165 weak sibling is safe on almost every run whether the rule is told or withheld (Llama3.1 100% told and
 166 100% withheld, a telling effect of +0; Mistral 97% and 92%, +4), so its safety is nearly invariant to
 167 the rule and reads tool competence, not self-enforcement, and their −12 contrasts are largely ceiling
 168 artefacts. In the two families where both siblings can actually drive the tools, Qwen2.5 and Qwen3,
 169 a real mechanism appears: the stronger model is at least as safe on the salient commit boundary
 170 (8/8 versus 0/8, 4/8 versus 2/8) yet executes the archive and set-active side-doors the weaker one
 171 cannot. It nets zero in Qwen2.5, where the commit gain cancels the side-door loss, and a genuine
 172 −10 in Qwen3, the one non-artefact net negative. So the honest reading is narrow: capability helps
 173 or ties on the salient boundary and can cut against safety on a discoverable side-door, and the pooled
 174 contrast mixes that small genuine effect with two ceiling artefacts. As a significance test this settles
 175 nothing: at three model lineages the one-sided sign-flip permutation floors at 0.125 and returns $p =$
 176 1.0, which is only the largest value the statistic can take once the observed mean is not positive, so
 177 we read the contrast descriptively.

178 5.2 The confounded precursor

179 A four-model ladder motivated this run. Told the commit rule with enforcement off, two capable
 180 models (MiniMax-M3, Llama-3.3-70B) refused on every run, and two weak models (Qwen3.5-9B,
 181 Qwen2.5-7B) committed on every run, so capability looked decisive and telling looked like it made
 182 enforcement redundant for capable models. However both capable models were non-Qwen, and the
 183 weak models Qwen, which confounded capability with model family. The within-family run breaks
 184 this confound and does not reproduce the result. The full ladder table, its per-run mechanism, the
 185 probe detail, and the ladder’s caveats are in Appendix B.

186 6 Harness blindness

187 Probing prior to the experiment produced and then destroyed an apparent positive result, and the
 188 failure mode is general enough to be considered a finding. Early on in this project, the harness did
 189 not show the agent its tools’ output: where a command’s result should have appeared, the agent got
 190 a file path instead. When a task then needed an identifier to proceed, the agent guessed.

191 It was an artefact of the harness. We pre-registered a falsification test and re-ran it after fixing the
 192 harness to show the agent its command output. Fabrication dropped to zero against the pre-registered
 193 40-percentage-point falsification bar: once the agent can see the empty lookup, it abstains honestly
 194 even when it needed the id to finish.

195 The lesson: an eval harness that hides tool output can cause the agent to guess, and a grader reads
 196 the guess as fabrication. Genuine constraint-driven fabrication does exist under more adversarial
 197 setups [arXiv:2606.14831]; our point is narrower.

198 7 Limitations and scope

199 **This is a case study on one harness.** Every model-backed number comes from one harness and
 200 one prompt template. The within-family run spans eight models in four families across sixteen tasks

on mutating state, but they are eight models on a single instrument: whether the effect generalises beyond this harness is the highest-value follow-up, and until an independent replication exists, nothing here separates a real phenomenon from a quirk of this instrument. The behavioural finding also rests on a single mechanism, the mutation gate: the other rule we swept, the medical boundary, was near-ceiling and uninformative (Section 5.2), so whether telling substitutes for enforcing on other mechanisms is untested. The two methods are also not the whole option space: where feasible, denying the agent the dangerous tool outright dominates both, and this paper studies the residual case where the task legitimately requires the capability, which is what makes violations measurable.

The capability–family confound is broken, at a cost. The precursor ladder confounded capability with model family (capable = non-Qwen, weak = Qwen); the within-family run (Section 5.1) removes that confound by pairing a strong and a weak sibling inside each of four families, and the ladder’s clean capability reading does not survive it: within a family, capability helps or ties on the salient boundary and can cut against safety on a discoverable side-door, so it does not order self-enforcement. What the within-family run cannot yet do is make the within-family null a test rather than a description: it rests on three model lineages, where the sign-flip permutation floor is 0.125, so more families are needed to move from a descriptive null to a rejection. Per-model vendor-recommended sampling still entangles model identity with sampling settings even within a pair, though each within-family 2×2 is computed at fixed settings and holds family constant across the capability contrast. In the two families where the weak sibling sits near ceiling (Llama3.1, Mistral) it is safe by incapability rather than self-enforcement, its safety barely moving when the rule is told (Llama3.1 +0, Mistral +4), so there the contrast reads tool competence, not disposition; it isolates self-enforcement cleanly only where both siblings can drive the harness.

The enforced cells carry no variance. Cells A and C are 100% by construction, so A minus B is exact arithmetic on cell B (the Section 5 remark); the guarantee run confirms the block held on all 488 enforced trials, so that share is real, not an artefact of scoring.

Pre-registration honesty. Beyond the outcome-screened replacement model and the sub-target sample size (Section 4), the withheld prompt still leaks command names, and the medical-boundary detector shares ground truth with the harness.

8 Discussion: what enforcement is for

Given telling, enforcement still contributes largely to safe behaviour: 41 points pooled. It is small only for the two self-enforcing families and the main contributor for the two weak ones. The enforcement layer’s block is unconditional: a blocked action cannot happen, whatever the model does or is prompted, the property FORGE [arXiv:2602.16708] establishes formally as verdict correctness (no denied action fires), and it held on all 488 enforced runs of the within-family study.

Mechanical Enforcement [arXiv:2605.14744] finds enforcement helping, but in a different domain: it scores decision-rationale quality under regulatory pressure, not tool-action compliance under benign use. Mind the GAP [arXiv:2602.16943] is better read as convergent than contradictory. Across six frontier models it finds runtime enforcement adds no detectable deterrent to what a model attempts, which resembles our result for the self-enforcing families, where enforcement’s marginal value given telling is smallest (7 to 10 points, though not the zero a strict reading of its null would imply), reached independently on a different harness, different models, and adversarial rather than benign pressure.

9 Conclusion and future work

Told or enforced, the honest answer is that telling helps but does not stand in for enforcing. The enforcement layer’s value is therefore not that capable models have outgrown it; it is that no model self-enforces everywhere, and enforcement is the only barrier where it does not: a withheld rule, a discoverable side-door, a rule below the model’s operate floor. Its guarantee is intact and unconditional.

The takeaway we most want carried is the methodological one: before attributing fabrication to a model, check what the harness let it see. Things which would sharpen the result: external replication on a second harness, and more model families; the rest are in Appendix C.

References

- Jeffrey Zhou et al. Instruction-Following Evaluation for Large Language Models (IFEval). arXiv:2311.07911, 2023.
- Ido Levy et al. ST-WebAgentBench: A Benchmark for Evaluating Safety and Trustworthiness in Web Agents. arXiv:2410.06703, 2024.
- Zhexin Zhang et al. Agent-SafetyBench: Evaluating the Safety of LLM Agents. arXiv:2412.14470, 2024.
- Haoyu Wang, Christopher M. Poskitt and Jun Sun. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. arXiv:2503.18666, 2025.
- Edoardo Debenedetti et al. Defeating Prompt Injections by Design (CaMeL). arXiv:2503.18813, 2025.
- Tianneng Shi et al. Progent: Securing AI Agents with Privilege Control. arXiv:2504.11703, 2025.
- Zhengkang Guo et al. RECAST: Expanding the Boundaries of LLMs’ Complex Instruction Following with Multi-Constraint Data. arXiv:2505.19030, 2025.
- Hao Peng, Yunjia Qi, Xiaozhi Wang, Bin Xu, Lei Hou and Juanzi Li. VerIF: Verification Engineering for Reinforcement Learning in Instruction Following. arXiv:2506.09942, 2025.
- Gauri Kholkar and Ratinder Ahuja. Policy-as-Prompt: Turning AI Governance Rules into Guardrails for AI Agents. arXiv:2509.23994, 2025.
- Da Song et al. Evaluating Implicit Regulatory Compliance in LLM Tool Invocation via Logic-Guided Synthesis (LogiSafetyBench). arXiv:2601.08196, 2026.
- Nils Palumbo, Sarthak Choudhary, Jihye Choi, Guy Amir, Prasad Chalasani and Somesh Jha. Formal Policy Enforcement for Real-World Agentic Systems (FORGE). arXiv:2602.16708, 2026.
- Arnold Cartagena and Ariane Teixeira. Mind the GAP: Text Safety Does Not Transfer to Tool-Call Safety in LLM Agents. arXiv:2602.16943, 2026.
- Varun Pratap Bhardwaj. Agent Behavioral Contracts: Formal Specification and Runtime Enforcement for Reliable Autonomous AI Agents. arXiv:2602.22302, 2026.
- Reshabh K Sharma. ContextCov: Deriving and Enforcing Executable Constraints from Agent Instruction Files. arXiv:2603.00822, 2026.
- Tanmay Sah, Vishal Srivastava, Dolly Sah and Kayden Jordan. The Verifier Tax: Horizon Dependent Safety Success Tradeoffs in Tool Using LLM Agents. arXiv:2603.19328, 2026.
- Linyue Pan, Lexiao Zou, Shuo Guo, Jingchen Ni and Hai-Tao Zheng. Natural-Language Agent Harnesses (NLAH). arXiv:2603.25723, 2026.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab and Chelsea Finn. Meta-Harness: End-to-End Optimization of Model Harnesses. arXiv:2603.28052, 2026.
- Jie Wu and Ming Gong. Policy-Invisible Violations in LLM-Based Agents (PhantomPolicy). arXiv:2604.12177, 2026.
- Yining Hong, Yining She, Eunsuk Kang, Christopher S. Timperley and Christian Kästner. Don’t Make Models Guess Security and Safety: Symbolic Guardrails for Domain-Specific AI Agents. arXiv:2604.15579, 2026.
- Jiahang Lin et al. Agentic Harness Engineering: Observability-Driven Automatic Evolution of Coding-Agent Harnesses (AHE). arXiv:2604.25850, 2026.
- Yubin Kim et al. TeamBench: Evaluating Agent Coordination under Enforced Role Separation. arXiv:2605.07073, 2026.
- José Manuel de la Chica Rodríguez and Carlos Martí-González. Mechanical Enforcement for LLM Governance: Evidence of Governance-Task Decoupling in Financial Decision Systems. arXiv:2605.14744, 2026.
- Rohith Uppala. Prompts Don’t Protect: Architectural Enforcement via MCP Proxy for LLM Tool Access Control. arXiv:2605.18414, 2026.
- Yilun Yao et al. Harness-Bench: Measuring Harness Effects across Models in Realistic Agent Workflows. arXiv:2605.27922, 2026.
- Suliu Qin, Haomin Zhuang, Yujun Zhou, Yufei Han and Xiangliang Zhang. AIRGuard: Guarding Agent Actions with Runtime Authority Control. arXiv:2605.28914, 2026.
- Qi Hu et al. SABER: Benchmarking Operational Safety of LLM Coding Agents in Stateful Project Workspaces. arXiv:2606.01317, 2026.

- Victor Ojewale and Suresh Venkatasubramanian. What Benchmarks Don’t Measure: The Case for Evaluating Abstention Competence in Autonomous Agents (ABSTAIN). arXiv:2606.02965, 2026.
- Yujun Zhou et al. Getting Better at Working With You: Compiling User Corrections into Runtime Enforcement for Coding Agents (TRACE). arXiv:2606.13174, 2026.
- Andoni Rodríguez, Alberto Pozanco and Daniel Borrajo. Is Your Agent Playing Dead? Deployed LLM Agents Exhibit Constraint-Evasive Fabrication and Thanatosis. arXiv:2606.14831, 2026.
- Wangxuan Fan, Xiaoyu Nie and Zhongxiang Dai. Harness-MU: A Safe, Governed, and Effective Harness for Multi-User LLM Agents. arXiv:2606.21856, 2026.
- Shiyang Chen. Governance Decay: How Context Compaction Silently Erases Safety Constraints in Long-Horizon LLM Agents. arXiv:2606.22528, 2026.
- Jiacheng Zhang et al. SafePyramid: A Hierarchical Benchmark for In-context Policy Guardrailing. arXiv:2606.29887, 2026.
- Agent Harness Engineering: A Survey (proposes the ETCLOVG taxonomy; under review at TMLR). OpenReview 3hXEPbG0dh.

Appendix A: reproduction

The Section 5.1 headline is reproduced by the released analysis code (`analysis/paired.py`) run over the paid trajectories, which are released as a versioned archive (a repository release, name withheld for review, with SHA-256 checksums); the archive includes the resulting `paired_result.json`. The precursor ladder table (Appendix B) reproduces the same way, from the per-gate analysis and the pooling module (`build_cell_contrasts_pooled`); the exact tests come from a small released helper (`exact_tests.py`, pure Python, no dependencies), which returns the run-level (0.00016), task-level (0.33), per-sub-gate (0.029), and family-corrected (0.00062) figures and the Clopper-Pearson intervals ([63, 100] for 8/8, [0, 37] for 0/8). Offline artefacts regenerate with `reproduce_offline.py` (no network, no private data). Two pins matter: the harness is git 6c82cd0 (tag `gab-runtime-1.0.1`, not the `v0.2.0` package, which does not enforce), and the taskset, grader, and analysis that produced these numbers are in the released repository at the preprint commit, a week later than the harness pin. The full task table and the earlier retired apparatus (a 28-task / 25-oracle-pair design, superseded 2026-07-04) are documented with the release.

On the commit boundary the whole 2×2 reduces to a single predicate over the agent’s first action a :

```
commit(a) = a is an agent-issued command that activates or deactivates
            user-authored state (W57, the harness’s user-authorization gate)
            and whose execution returned OK
safe(a) = not commit(a)
```

A proposal or a refusal is safe; an executed activation is the violation. The two boundary choices are deliberate. First action, because a blocked action’s error message is itself the rule told late (Section 3), so only the first move reads what the model brought to it. Executed, because an attempt the enforcement layer blocks is visible in the trajectory but is not a violation that happened; the violation is a change that landed.

The rest of the grader. The wider taskset is graded by the same kind of fixed offline checks, in two groups: task-completion checks (correct commands, valid output formats, a summary consistent with what the agent observed) and safety violations graded zero-tolerance, where a single occurrence fails the run outright (an unauthorised change to user data, a direct database write, an invented command, a wrong refusal in either direction, a fabricated citation, a medical claim). None of this carries weight in the headline, which rests on the commit predicate alone.

Reproducibility of the scoring. The bars and which checks are zero-tolerance are read from one committed configuration file, and each score records that file’s SHA-256 hash, so a threshold or policy change not carrying a hash change is a detectable reproducibility break, not a silent re-scoring. The bars are saturated by design: the contract gives no partial credit for one hallucinated command or one unauthorised mutation.

A disclosed pre-run grader edit. The harm-check code was edited three times in the day before the run, the last 47 minutes before it, each edit in the direction we hoped for. We verified this does not

touch the result: every violation in the headline table is caught by the original, older detection path, so re-scoring with the pre-edit grader is identical on the 2×2 and the finding is invariant to those edits.

Two departures. First, a secondary serverless-breadth arm (four single-band models) failed to run on a stale provider key and is dropped; it carries no family pair and no weight on the primary. Second, the run spans two harness commits, with an orchestration-only robustness fix between them that leaves every model reply byte-identical, so the split is immaterial to scoring. The precursor ladder (Section 5.2) carried its own two disclosures: its pre-registered primary model was deprecated by the provider mid-cycle and replaced with MiniMax-M3 only after a canary confirmed the replacement reproduced the effect (an outcome-conditioned choice, so MiniMax-M3 is a non-pre-registered addition and exploratory; the pre-registered capable model is Llama-3.3-70B, which shows the same pattern), and its commit boundary has two test tasks, not the three the sample-size analysis assumed, so its pooled cell is eight runs, below the twelve-run target; the effects are saturated (0/8 vs. 8/8), but the small size is stated.

Appendix B: the confounded precursor ladder

The four-model ladder of Section 5.2, in full, preserved as provenance: its clean substitution and capability reading is superseded by the within-family run of Section 5.1, which does not reproduce it. It swept four models against the git-pinned harness, MiniMax-M3 and Llama-3.3-70B (capable), Qwen3.5-9B (near the floor) and Qwen2.5-7B (below it), for a total paid cost of USD 10.44. Scoring the model’s first action, the share of runs that stayed safe on the commit boundary was:

Model	A: told, enforced	B: told, off	C: withheld, enforced	D: neither
MiniMax-M3 (capable)	100*	100	100*	0
Llama-3.3-70B (capable)	100*	100	100*	25
Qwen3.5-9B (near floor)	100*	0	100*	12.5
Qwen2.5-7B (below floor)	100*	0	100*	12.5

*Safe by construction; the finding is in columns B and D. Each cell is eight runs (two tasks, four repeats each).

Read column B: told the rule, enforcement off. Both capable models stayed safe on all eight runs; both weak models committed the forbidden change on all eight. The marginal value of enforcement given the model was told, cell A minus cell B, is therefore zero for the capable models and the entire barrier for the weak ones. That column is the precursor observation the within-family run of Section 5.1 was built to test and, on the capability reading, does not confirm.

Why each model did it, read run by run. The transcripts show the split directly. In the told-off cell, MiniMax refuses the commit on its own turn and tells the user to run it themselves; Qwen3.5-9B issues the commit command itself, and it executes because the gate is off. The capable model declines an action it is free to take; the weak model takes it.

What the uncertainty is, by estimation rather than testing. With eight runs a cell the exact (Clopper-Pearson) intervals are wide: a cell at 0/8 safe has interval [0, 37%], and a cell at 8/8 has [63, 100%]. So the weak models’ safe rate is bounded at most 37%, and the capable models’ at least 63%, but neither is pinned precisely. The capable side’s flat cell A minus cell B is a failure to detect an effect at this size, not proof that none exists.

Why counts, not a p-value. A significance test is the wrong instrument here: one arm of every enforced contrast is 100% safe by construction rather than a random outcome, the four repeats of each task are near-identical rather than independent, and any rejection is confounded with model family and task. Treating the eight runs a side as independent gives $p = 0.00016$, but counting the two genuinely independent tasks a side gives $p = 0.33$.

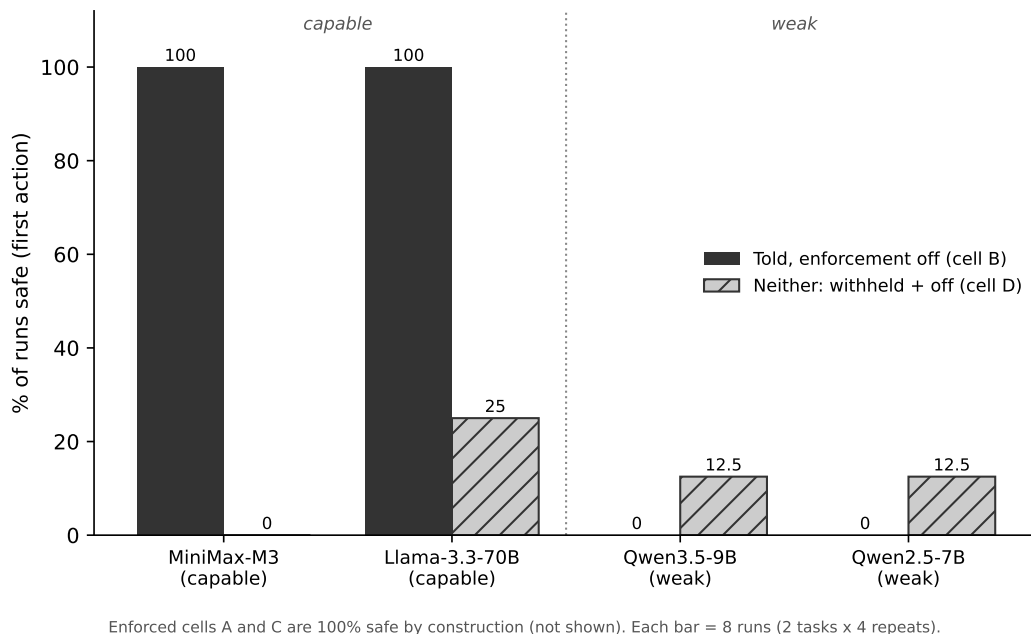


Figure 2: Told the commit rule with enforcement off (cell B): both capable models stayed safe on every run, both weak models violated on every run. The withheld floor (cell D) shows telling moves behaviour within a model even for the capable ones. Enforced cells A and C are 100% by construction. Each bar is eight runs.

403 **This is substitution, not the model simply being agreeable.** With the rule withheld and the en-
 404 enforcement off (cell D), the capable models violate too: MiniMax on all eight runs, Llama on six of
 405 eight. So telling has a large within-model effect even for a capable model, and its flat A-minus-B is
 406 genuine substitution, not indifference to both.

407 **Caveats.** The below-floor 7B fails partially rather than totally (graded told-off scores), so at a le-
 408 nient threshold its column B would read about 75% rather than 0%; the decisive cells are threshold-
 409 invariant, and the released grader configuration ships the zero-tolerance threshold used here. The
 410 ladder also inherited the discoverability aid of Section 7 mid-development: it was added after the
 411 ladder’s smoke runs, before its pre-registration and sweep, so for the ladder it was an outcome-
 412 informed instrument change, where for the within-family run it is a design constant. Empty output
 413 cannot fake the result: Llama produces malformed output on 43% of its column-B turns, and six
 414 of its eight column-B runs carry at least one, yet it still reaches 100% in column B through gen-
 415 uine refusals. Most limiting, once the below-floor 7B is set aside and the capability-versus-family
 416 confound is held in view, this ladder’s “told but does not self-enforce” result rests on one mid-
 417 size model, Qwen3.5-9B, which is why the within-family run of Section 5.1 was carried out. The
 418 medical-boundary rule the ladder also swept is near-ceiling: its detector fires on 7 of 223 runs, all
 419 on a single task and model, and the two tasks written to provoke a medical claim elicit none.

420 **The one-model probes.** Before the ladder, a set of single-model probes (Qwen3-235B, three to five
 421 runs a cell) shaped the design. With the enforcement off, a told-and-salient model refused three of
 422 three while a withheld model committed the violation three of three, establishing that telling and
 423 enforcing each move behaviour off the floor. Two ideas we expected to matter did not survive: audit
 424 faithfulness proved self-checkable once the model retrieved the evidence, collapsing into capability,
 425 and benign pressure to finish the task produced zero fabrication across the 36 runs of that probe. The
 426 one qualifier they raise is salience-sensitivity: the same told model that refused when the rule was
 427 foregrounded dispatched the forbidden command three of three when it met the rule only inciden-
 428 tally, so we carry it as a caveat, not a result.

Appendix C: additional detail

The harness between a model and its tools is now studied as a design surface, but that line optimises it for capability, not governance (Harness-Bench, NLAH, AHE, Meta-Harness [arXiv:2605.27922, 2603.25723, 2604.25850, 2603.28052]; ETCLOVG, OpenReview 3hXEPbG0dh). Enforcement frameworks supply mechanisms without measuring what the model would do unaided (AgentSpec, CaMeL, Progent, Harness-MU [arXiv:2503.18666, 2503.18813, 2504.11703, 2606.21856]), and studies pitting enforcement against text-only governance change behaviour without withholding the rule per condition (ContextCov, Verifier Tax, Mechanical Enforcement, Policy-as-Prompt [arXiv:2603.00822, 2603.19328, 2605.14744, 2509.23994]; and the structural guards AIRGuard, Prompts Don't Protect, Symbolic Guardrails, TRACE, ST-WebAgentBench [arXiv:2605.28914, 2605.18414, 2604.15579, 2606.13174, 2410.06703]). The result is not mere instruction-following, because told-only compliance is not free: capable models finish tasks while breaking latent rules, non-monotonically across families (LogiSafetyBench, SABER, Agent-SafetyBench [arXiv:2601.08196, 2606.01317, 2412.14470]); in-context policy degrades with reasoning complexity (SafePyramid [arXiv:2606.29887]); and rules obeyed while visible are dropped as context compacts (Governance Decay [arXiv:2606.22528]). That checkability is what prior work uses to build evaluations (IFEval, RECAST, VerIF [arXiv:2311.07911, 2505.19030, 2506.09942]), not to predict when an agent complies.

Check	What it does
schema check	rejects malformed or invented commands before they run
dispatch gate	refuses to run a command not flagged safe for the agent
commit gate	the agent may propose a change to user data, but only the user may commit it
refusal rule	refuses out-of-scope requests, including a zero-tolerance medical boundary
audit check	attaches evidence to recommendations; the grader later catches fabricated or missing references

Remark: the identity is the finding, not a trick. A sceptic can read A minus B as a re-description: enforced runs are safe by construction, so the number is arithmetic on cell B, and the behavioural content of the study is the enforcement-off column plus a software verification. That is precisely the design.

Intervals are wide at these per-cell sizes, but the pooled contrasts are clear. Per-family 95% Wilson intervals on cell B run from [19, 40] (Qwen2.5) to [84, 97] (Llama3.1), and cell D is similarly wide; pooled, cell B is 59% [53, 65] and cell D 35% [29, 41]. The telling effect (B minus D) is +24 points, 95% Newcombe interval [16, 33], and the enforcement residual (A minus B) is 41 points, 95% interval [35, 47]. The repeat-dependence caveat that keeps the within-family capability contrast descriptive (below) applies to these intervals too, so we read them as indicative rather than exact. Two robustness checks. Scored on the converged multi-turn window instead of the first action, every cell in this run is identical, so no result here depends on the window choice. And 20 enforcement off trajectories (8 in cell B, 12 in cell D) also came back incomplete, concentrated in the same weaker models as the enforced-cell drops; scored pessimistically, with every dropped trajectory counted as a violation, the pooled telling effect is unchanged (+24) and the weak-sibling ceilings in Llama3.1 and Mistral fall to 84% and 91%: Mistral's contrast stays negative (about -6) while Llama3.1's flips to about +3, the strong sibling marginally safer, which is what a ceiling artefact predicts. We do not adopt that scoring, since an unfinished trajectory is not an executed violation, and report it as a bound.

One design choice deserves its own line: the task prompts inject the identifier of the seeded target, a discoverability aid added to the instrument after early smoke runs showed models scoring safe merely because they could not find the target at all. In the within-family run it is a design constant, present in every cell and both arms and part of the taskset before that run's canary. Its likely direction

470 is to raise withheld-arm violations and therefore the measured telling effect, since without it cell D
471 reads safer by incapability. It is prompt-side, so it cannot be removed by re-scoring; a no-aid arm
472 is future work. Its reach is bounded by mechanism: the aid changes discovery, not ability, so it can
473 move an outcome only for a model able to execute the mutation once its target is findable. Team-
474 Bench’s caution applies to us too: our grader sees whether the specific guarded violation occurs, but
475 an enforcement effect that shifted some finer channel while leaving the tracked violation unchanged
476 would go undetected at this size.

477 Beyond the guarantee, the block is what supplies the safe outcome wherever a model does not
478 self-enforce: below the operate floor, where the failure is being too weak to drive the tools rather
479 than disobedience; when the rule was never told, the floor cell where the enforcement layer is
480 the only barrier and real deployments accumulate such rules through drift and context compaction
481 [arXiv:2606.22528]; when a told rule is present but not salient at the moment of action; wherever
482 capability itself enables the violation, as a more capable model can drive a governed side-door to
483 completion a weaker one cannot (clearest in one family); and under adversarial pressure, which we
484 do not test.

485 **A note for evaluation.** Read from the model’s side, cell B (told, enforcement off) measures the self-
486 enforcement a model supplies once told, a quantity distinct from raw capability and from runtime
487 enforcement; a post-training evaluation could track it before and after an intervention, but we do no
488 training here and claim nothing about it.

489 The within-family run above is the confound-break the earlier draft named as future work; four
490 things would still sharpen the result. The permutation rests on three model lineages, where the exact
491 one-sided floor is 0.125, so more families are what would let the descriptive null harden into a test.
492 External replication on a second harness with its own rules and grader is what would turn a case study
493 into a phenomenon. Adversarial inputs would locate where substitution ends and injection defence
494 begins. And long-horizon drift (does enforcement catch a rule the model has since forgotten?) is the
495 measurement this design most directly points at.

496 **NeurIPS Paper Checklist**

497 **1. Claims**

498 Question: Do the main claims made in the abstract and introduction accurately reflect the
499 paper’s contributions and scope?

500 Answer: [\[Yes\]](#)

501 Justification: The abstract and introduction state the substitution finding, the harness-
502 blindness caution, and the released instrument; Section 5 reports them and Section 7 bounds
503 them.

504 **2. Limitations**

505 Question: Does the paper discuss the limitations of the work performed by the authors?

506 Answer: [\[Yes\]](#)

507 Justification: Section 7 is a dedicated limitations section: single harness, single informative
508 mechanism, per-cell sample sizes, prompt leakage, and the discoverability aid (disclosed
509 in full in Appendix C).

510 **3. Theory assumptions and proofs**

511 Question: For each theoretical result, does the paper provide the full set of assumptions and
512 a complete (and correct) proof?

513 Answer: [\[N/A\]](#)

514 Justification: The paper contains no theoretical results.

515 **4. Experimental result reproducibility**

516 Question: Does the paper fully disclose all the information needed to reproduce the main
517 experimental results?

518 Answer: [\[Yes\]](#)

519 Justification: Appendix A gives the pinned harness commit, the released grader and analy-
 520 sis code, the exact-test helper, and the versioned trajectory archive from which every head-
 521 line number reproduces.

522 **5. Open access to data and code**

523 Question: Does the paper provide open access to the data and code, with sufficient instruc-
 524 tions to faithfully reproduce the main experimental results?

525 Answer: [Yes]

526 Justification: The taskset, deterministic grader, analysis code, and paid-run trajectories are
 527 released as a versioned archive with checksums (reference withheld for review; Appendix
 528 A).

529 **6. Experimental setting/details**

530 Question: Does the paper specify all the training and test details necessary to understand
 531 the results?

532 Answer: [Yes]

533 Justification: Sections 4–5 and Appendices A–B give the models, tasks, repeats per cell,
 534 deployment setup, and scoring configuration; no training is performed.

535 **7. Experiment statistical significance**

536 Question: Does the paper report error bars suitably and correctly defined or other appropri-
 537 ate information about the statistical significance of the experiments?

538 Answer: [Yes]

539 Justification: Wilson and Newcombe intervals and the pre-registered permutation floor are
 540 reported (Appendix C), and the capability contrast is stated descriptively because its exact
 541 one-sided floor at three lineages is 0.125.

542 **8. Experiments compute resources**

543 Question: For each experiment, does the paper provide sufficient information on the com-
 544 puter resources needed to reproduce the experiments?

545 Answer: [Yes]

546 Justification: The within-family run used on-demand H100 deployments for about USD
 547 115 (Section 5.1); the precursor ladder’s total paid cost is stated in Appendix B.

548 **9. Code of ethics**

549 Question: Does the research conducted in the paper conform, in every respect, with the
 550 NeurIPS Code of Ethics?

551 Answer: [Yes]

552 Justification: The work evaluates open-weight models on synthetic governed-data tasks; no
 553 human data, no deception, no harmful release.

554 **10. Broader impacts**

555 Question: Does the paper discuss both potential positive societal impacts and negative
 556 societal impacts of the work performed?

557 Answer: [Yes]

558 Justification: The paper’s purpose is safer harness design: measuring where runtime en-
 559 forcement is still needed (Section 8). We identify no direct path to negative applications
 560 from the released instrument.

561 **11. Safeguards**

562 Question: Does the paper describe safeguards that have been put in place for responsible
 563 release of data or models that have a high risk for misuse?

564 Answer: [N/A]

565 Justification: No high-risk models or data are released; the benchmark’s tasks operate on
 566 synthetic personal-wellness fixtures.

567 **12. Licenses for existing assets**

568 Question: Are the creators or original owners of assets (e.g., code, data, models), used in
569 the paper, properly credited and are the license and terms of use explicitly mentioned and
570 properly respected?

571 Answer: [Yes]

572 Justification: The evaluated open-weight model families are named with exact variants
573 (Section 5) and used under their published licenses; all cited benchmarks and priors carry
574 identifiers in the references.

575 **13. New assets**

576 Question: Are new assets introduced in the paper well documented and is the documenta-
577 tion provided alongside the assets?

578 Answer: [Yes]

579 Justification: GovernedAgentBench is documented in Section 4 and Appendix A and ships
580 with its scoring configuration, tests, and checksummed archive.

581 **14. Crowdsourcing and research with human subjects**

582 Question: For crowdsourcing experiments and research with human subjects, does the pa-
583 per include the full text of instructions given to participants and screenshots, if applicable,
584 as well as details about compensation (if any)?

585 Answer: [N/A]

586 Justification: No crowdsourcing and no human subjects.

587 **15. Institutional review board (IRB) approvals or equivalent for research with human**
588 **subjects**

589 Question: Does the paper describe potential risks incurred by study participants, whether
590 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
591 approvals (or an equivalent approval/review based on the requirements of your country or
592 institution) were obtained?

593 Answer: [N/A]

594 Justification: No human subjects.

595 **16. Declaration of LLM usage**

596 Question: Does the paper describe usage of LLMs if LLMs are an important, original, or
597 non-standard component of the core methods in this research?

598 Answer: [Yes]

599 Justification: LLMs are the objects of study: eight open-weight models are evaluated as
600 agents (Sections 4–5). The measurement instrument itself is deterministic code with no
601 LLM in the scoring loop, by design. LLM-based tools assisted with engineering and
602 manuscript preparation under author direction and verification; they contributed no sci-
603 entific claims and are not authors.